

Problem H. Who wants to be a Millionaire?

Source file name: Millionaire.c, Millionaire.cpp, Millionaire.java, Millionaire.py
Input: Standard
Output: Standard

In the popular game show, “Who wants to be a Millionaire?,” contestants are given 15 questions, in increasing order of difficulty. The cash a contestant wins is dependent on the number of questions they answer correctly. An incorrect response immediately ends the game, and the contestant either wins no money, or a reduced amount of money. Alternatively, a contestant could choose to cash out at any time and get all the winnings for their correctly answered questions.

In this problem, we’ll analyze an easier version of the game where an incorrect response to a question automatically means no winnings.

In this version of the game, a contestant will receive up to n questions in series. Let the questions be numbered 1 through n . For each question, there is a probability that the contestant will answer the question correctly. After listening to each question, the contestant can choose to answer the question or to cash out. If the previous question was answered correctly (and it wasn’t the last question), the contestant gets to hear the next question, at which point they have the same choice to make, i.e., choose to answer the question or cash out.

To help the contestant, they get to “ask an expert” for help on two questions, i.e., the contestant can get help from an expert on two questions out of the n questions. For each question, there is an alternate probability the expert will correctly answer the question (and this is guaranteed to be strictly higher than the probability the contestant will answer the question correctly). On any two questions out of the n total questions, the contestant can choose this option once they’ve decided to answer the question.

For each of the n questions, there is a dollar amount assigned. That is the amount the contestant wins if that particular question was the last question they answered correctly. For example, if after hearing question #8, a contestant decides to not answer it and cash out, the amount of money they will win is the amount of money associated with question #7, the last question they answered correctly. Recall that if they answer their last question incorrectly, the contestant wins no money.

Given the number of questions in the game, the dollar amount assigned to each of the questions, the probability that the contestant will correctly answer each of the questions, and the probability that the expert will correctly answer each of the questions, determine the maximum expected winnings of the contestant, if they play optimally.

Input

The first input line contains a single integer, n ($1 \leq n \leq 100$), indicating the number of questions in the quiz show.

The second input line contains n integers, $w_1, w_2, \dots, w_n$. w_i equals the total winnings for answering question i correctly and then stopping the game. It is guaranteed that $0 < w_1 < w_2 < w_3 < \dots < w_{n-1} < w_n \leq 10^9$.

The third input line contains n floating point numbers, $p_1, p_2, \dots, p_n$. p_i equals the probability of the contestant answering question i correctly. It is guaranteed that each of these will be specified to exactly 2 decimal places and $0 \leq p_i < 1$.

The fourth input line contains n floating point numbers, $q_1, q_2, \dots, q_n$. q_i equals the probability of the expert answering question i correctly. It is guaranteed that each of these will be specified to exactly 2 decimal places and $p_i < q_i \leq 1$.



Output

Print the maximum expected winnings of the contestant, if they play optimally. Any answer within an absolute or relative error of 10^{-6} of the correct answer will be accepted.

Example

Input	Output
2 10 100 0.25 0.01 0.99 1.00	99.0
3 100 101 102 0.90 0.80 0.70 1.00 0.81 0.71	100.0

Explanation

For the first Example, the best strategy is to ask the expert both questions.

For the second Example, the best strategy is to ask the expert the first question and then stop.